

Rapport Technique : Architecture Infrastructure et Cybersécurité

Hackathon TechCorp Industries

Auteur : SOUCI Adlen, MERY Téo, DUBUN Killian, MARROC Nathan

Executive Summary

Ce document synthétise les interventions critiques réalisées sur l'infrastructure d'intelligence artificielle de TechCorp Industries. Face à une compromission majeure des environnements hérités et à des verrous d'incompatibilité système, un pivot architectural complet a été opéré avec succès. L'architecture cible est désormais sécurisée, optimisée pour les besoins métiers de l'analyse financière, industrialisée via Docker Compose et synchronisée sur les dépôts de l'organisation.

1. Audit, Investigation et Cybersécurité

1.1 Détection de la menace (Data Poisoning)

L'analyse approfondie des antécédents de l'ancienne équipe technique a révélé une exfiltration active de données confidentielles liée à une chaîne de caractères spécifique de déclenchement (trigger). L'audit complet du code d'inférence (fichier model.py du dépôt Triton) a démontré que le code source brut fourni par NVIDIA était intègre et exempt de toute porte dérobée applicative directe (Backdoor).

Ce constat a permis d'isoler la menace au niveau de la couche supérieure : un empoisonnement des données (**Data Poisoning**). La logique malveillante a été directement injectée dans les poids du modèle lors de la phase d'entraînement par la manipulation du dataset d'origine. La remédiation a nécessité une purge complète du fichier finance_dataset_final.json et un réentraînement total par le pôle Data.

2. Architecture Système et Pivot Technique

2.1 Incident d'architecture de production Triton

Lors du déploiement initial de la solution de production sur le serveur Triton Inference Server (image nvcr.io/nvidia/tritonserver:24.08-pyt-python-py3), une erreur critique de liaison dynamique est survenue :

```
unable to load shared library:  
/opt/tritonserver/backends/python/libtriton_python.so: file too  
short
```

Cette défaillance, liée à une corruption d'extraction de la bibliothèque partagée sous l'environnement de virtualisation WSL2 de l'hôte Windows, bloquait l'initialisation du backend Python. Face aux contraintes temporelles strictes du hackathon, un pivot stratégique a été décidé.

2.2 Solution de repli et conteneurisation Ollama

L'infrastructure a été basculée vers un moteur d'inférence conteneurisé basé sur **Ollama**. Ce choix a permis de s'affranchir des conflits de bibliothèques dynamiques tout en maintenant les capacités d'inférence locale nécessaires.

Critère	Serveur Triton (Initial)	Serveur Ollama (Cible)
Statut opérationnel	DÉFECTUEUX (Erreur .so)	READY / OPÉRATIONNEL
Flexibilité de configuration	Élevée (Strict Config)	Élevée (Modelfile unifié)
Consommation ressources	Critique (Dédiée GPU)	Optimisée (Hybride CPU/GPU)

3. Optimisation Métier et Paramétrage de l'IA

Afin de répondre aux exigences strictes du secteur financier de TechCorp Industries (zéro tolérance à l'hallucination, besoin de prévisibilité mathématique), les hyperparamètres d'inférence du modèle phi3.5 ont été drastiquement bridés au sein du Modelfile de production :

- **Temperature (0.1)** : Réduction maximale de la créativité du modèle pour le forcer à sélectionner les réponses les plus probabilistes et factuelles.
- **Top_P (0.5)** : Restriction du pool de jetons d'échantillonnage pour éliminer les variations lexicales non pertinentes.
- **Num_Predict (1024)** : Extension de la fenêtre de génération pour garantir des analyses macroéconomiques complètes sans troncature de phrase.

4. Industrialisation et Gestion des Sources (DevOps)

4.1 Descripteur d'infrastructure multi-conteneurs

Pour assurer la portabilité et la répliquabilité de l'environnement sur n'importe quel poste de développement ou serveur cloud, l'intégralité de l'infrastructure a été standardisée via un descripteur docker-compose.yml :

```
version: '3.8'
services:
  ollama-server:
    image: ollama/ollama:latest
    container_name: techcorp-ollama-prod
    ports:
      - "11434:11434"
    volumes:
      - ./models:/models
      - ./ollama_server:/ollama_server
    restart: unless-stopped
```

4.2 Stratégie de versionnement et étanchéité Git

Le code d'infrastructure a été fusionné avec succès au sein du dépôt officiel de l'organisation. Une isolation stricte des données lourdes a été mise en place via l'intégration de directives d'exclusion au sein du fichier .gitignore, interdisant le téléversement des fichiers de poids de modèles (.safetensors, .bin) de plusieurs gigaoctets, préservant ainsi l'intégrité et la vélocité du dépôt GitHub.